Construction Progress Dashboard

End-to-End DevOps Lifecycle: Automated Quality Gates, Docker Containers & Ansible CD

Author: Ganesh Dahiphale | DevOps Final Capstone (Week 15) | Release v1.1.0

Problem Statement & Strategic Engineering Objectives

The Construction Industry Problem

- Severe visibility gap between jobsite field engineers and office executives.
- Progress logged in paper diaries or isolated messaging threads.
- Milestone delays and blocked activities discovered weeks after occurrence.
- Manual, ad-hoc server deployments with zero rollback capability.
- Fragile releases prone to production outages and regression defects.

DevOps Engineering Objectives

- Deliver a robust full-stack Spring Boot 3 platform for real-time progress logging.
- Enforce automated quality assurance via headless Selenium E2E browser tests.
- Package production binaries into minimal, non-root Docker runtime images.
- Orchestrate idempotent server configuration and deployments with Ansible.
- Guarantee sub-minute automated rollback upon synthetic health gate failure.
- Eliminate manual release friction through a 100% automated CI/CD pipeline.

Application Scope & Production MVP Features (v1.0.0 & v1.1.0)

Field Task Operations

- Task logging: title, project, due date, completion %.
- Automatic completion business rule: setting status COMPLETED locks progress to 100%.
- Audit trail: tracks updatedBy username and lastUpdated timestamp.
- Multi-parameter search filtering by project and status.

Executive Insights & Alerts

- Consolidated Executive KPI Dashboard (/dashboard).
- Real-time counters: Total, Completed, In-Progress, Blocked, Delayed.
- Dynamic Status Drill-Down (/status).
- Automated Alert Engine (/alerts) identifying overdue tasks against system date.

Security & Governance

- Spring Security 6 Role-Based Access Control (RBAC).
- Roles: ADMIN (Site configs), MANAGER (Dashboards), ENGINEER (Field logs).
- Salte BCrypt password encryption (strength 10).
- Synchronizer token CSRF form protection.
- Synthetic REST health endpoint (/health).

Multi-Tier Enterprise Architecture & Relational Domain Model

Architectural Layers (Spring Boot 3)

• Client Tier: Modern web browsers & automated Selenium Headless Chrome.

• Security Tier: DelegatingFilterProxy & SecurityFilterChain with BCrypt auth.

• Presentation Tier: Spring MVC controllers rendering Thymeleaf 5 HTML.

• Service Tier: Domain rules (KPI calculations, auto-100% completion, alerts).

• Persistence Tier: Spring Data JPA Repositories with dynamic Specifications.

• Database Engine: Relational engine (H2 dev profile / PostgreSQL ready).

Domain Entity Relationships (ERD)

• PROJECT (1) ——< TASK (N): One project aggregates multiple site tasks.

• USER (1) ——> TASK (Audits): Tracks lead engineers and last mutation.

• Project Entity: ID, Name, Description, Location, Start Date, Target Date.

• Task Entity: ID, Title, Status (Enum), PercentComplete (0-100), DueDate.

• User Entity: ID, Username, Password (BCrypt Hash), Role (Enum), Enabled.

Agile Scrum Methodology & Sprint Delivery Milestones

Sprint Trajectory (Weeks 4 – 15)

• Sprint 1 (Weeks 4–6): Domain foundation, field CRUD, MVP v0.2.0 baseline.

• Sprint 2 (Week 7): Spring Security 6 RBAC, Admin portal, Release v1.0.0.

• Sprint 3 (Weeks 8–10): Jenkins Pipeline as code, Selenium testing & Quality Gate.

• Sprint 4 (Weeks 11–12): Multi-stage Docker, registry publishing, Docker CD.

• Sprint 5 (Weeks 13–14): Ansible provisioning, idempotency, automated rollback.

• Sprint 6 (Week 15): End-to-end Git-to-Ansible workflow & Release v1.1.0.

Agile Quality Governance

• Product Backlog: Tracked user stories B-01 to B-14 in docs/BACKLOG.md.

• GitHub Issues: Every task mapped to discrete issues with Acceptance Criteria.

• Definition of Done (DoD): Code quality, 100% test pass, PR review, CI green.

• Immutable Releases: v1.0.0 (MVP milestone) and v1.1.0 (DevOps milestone).

• Artifact Attachments: Production WAR attached to GitHub Releases.

Version Control Governance: Gitflow, Traceability & Merge Resolution

Gitflow Branching Architecture

• Perpetual Branches: main (production releases) and develop (integration).

• Ephemeral Branches: feature/N-*, chore/*, docs/* created off develop.

• Pull Requests: Enforce peer reviews, status checks, and non-fast-forward merges.

• Traceable Commits: Conventional Commits with mandatory issue IDs (#N).

• Security Policy: Zero credentials committed; git push --force strictly disabled.

Real Merge Conflict Resolution (Week 6)

• Scenario: Feature A (#7) and Feature B (#8) branched from commit c90f7c7.

• Conflict: Both branches modified navbar.html and README.md concurrently.

• Detection: GitHub CLI flagged PR #10 as DIRTY and CONFLICTING.

• Resolution: Performed local merge develop, unified navigation links for both features.

• Verification: Tested locally, pushed merge resolution commit, successfully merged PR #10.

Declarative CI/CD Pipeline Architecture (Jenkinsfile)

Ten Automated Pipeline Stages

- 1. Checkout: Metadata extraction (APP_VERSION, IMAGE_TAG).
- 2. Compile: Clean compilation of 30 Java classes via Maven.
- 3. Unit Tests: 58 JUnit 5 tests covering models, services, controllers.
- 4. Package: Produces 53.84 MB production WAR artifact (dashboard.war).
- 5. Selenium Quality Gate: Headless Chrome browser user journeys.
- 6. Docker Build: Multi-stage container packaging.
- 7. Docker Push: Pushes versioned tag & :latest to registry.
- 8. Ansible Deploy: Provisions node & restarts container via systemd.
- 9. Verify Node: Polling HTTP /health and UI version badge.
- 10. Promote: Promotes image tag to :stable upon 100% verification.

Pipeline Design Principles

- Deterministic Tagging: <version>-b<build>-<sha> (e.g., 0.0.1-SNAPSHOT-b3-73fa1bc).
- Fail-Fast Quality Gates: Failed tests immediately abort downstream publishing.
- Artifact Archiving: Archives JUnit XML, deploy-info.txt, and Ansible logs.
- Security Isolation: Injects secrets via Jenkins Credentials Store (never printed).

Continuous Testing: Automated Selenium Headless Quality Gate

Headless Chrome User Journeys

- Journey 1: Form login, session cookies, and RBAC authorization restrictions.
- Journey 2: Task creation, field validation, and 100% auto-completion logic.
- Journey 3: Executive dashboard filtering, KPI counter assertions, and alert views.
- Implementation: Page Object Model (POM) pattern for UI maintainability.
- Runtime Flags: --headless=new, --no-sandbox, --disable-dev-shm-usage.

Defect Blockade Demonstration (Week 10)

- Baseline Run (Build #1): Green, 100% test pass, deployment executed.
- Defect Injection (Build #2): Injected UI form assertion defect.
- Blockade Outcome: Selenium caught failure in 88.2s; pipeline halted.
- Production Protection: Docker build and Ansible deploy stages were SKIPPED.
- Defect Remediation (Build #3): Fixed defect; pipeline succeeded and promoted.

Hardened Containerization: Multi-Stage Dockerfile & Registry

Multi-Stage Dockerfile Design

• Stage 1 (Builder): maven:3.9-eclipse-temurin-17 packages the application.

• Stage 2 (Runtime): eclipse-temurin:17-jre-jammy minimal runtime image.

• Layer Caching: pom.xml dependencies copied first; cuts rebuilds to 8.4s.

• Least Privilege: Dedicated unprivileged user appuser (UID 10001).

• Healthcheck: Built-in Docker HEALTHCHECK probe on http://localhost:8080/health.

• Network Mapping: Published on host port 8082 -> container port 8080.

Registry & Image Lifecycle

• Build Artifact: construction-dashboard:<version>-b<build>-<sha>.

• Dual Tagging: Each push updates both unique build tag and :latest.

• Promotion Strategy: Only deployed and verified images are tagged :stable.

• Registry Distribution: Deployed via local distribution registry (port 5000) and Docker Hub.

Configuration Management: Modular Ansible Architecture & Roles

Ansible Role Hierarchy

• common: System utilities, apt cache update, Asia/Kolkata timezone.

• docker: Upstream Docker CE engine, daemon.json log rotation (10MB x 3).

• app_user: Dedicated system user appdeploy with Docker group access.

• app_dirs: Standard structure /opt/construction-dashboard/{config,logs,releases}.

• firewall: UFW policy (default-deny inbound; allows port 22 and port 8082).

• app_config: Deploys app.env (mode 0600) and systemd service unit.

• deploy: Container pull, health polling, rollback, and release logging.

systemd Supervisor & Host Hardening

• Systemd Unit: construction-dashboard.service manages container lifecycle.

• Auto-Restart: Restart=always recovers container if terminated unexpectedly.

• File Permissions: App configs locked to 0750/0600 appdeploy:docker.

• Standards Compliance: 0 errors in yamllint and passed strict ansible-lint production profile.

Production Reliability: Strict Idempotency & Automated Rollback

Strict Idempotency Demonstrated

- Execution 1: Provisioned server prerequisites (changed=20).
- Execution 2: Re-ran entire playbook with zero modifications.
- Play Recap: ok=37 changed=0 unreachable=0 failed=0.
- Significance: Proves infrastructure is declaratively defined without side-effects.
- Drift Reconciliation: Deliberate manual drift automatically corrected by Ansible.

Health Gates & Atomic Auto-Rollback

- Architecture: deploy role implements an Ansible block / rescue construct.
- Health Polling: Polls /health every 5s for up to 25 retries (125s max).
- Failure Scenario: Deployed broken-demo image with misconfigured port.
- Rollback Action: Rescue caught failure, reverted to previous.txt, restarted container.
- Restoration Time: Service restored in < 48s; audit logged to history.log.

Empirical Results: Full Pipeline Timeline & Performance Metrics

Final Build #3 Timeline Breakdown

- Source Compilation (Maven): 5.4 seconds (30 Java classes).
- Unit & Integration Tests: 18.8 seconds (58 test cases, 0 failures).
- WAR Packaging: 5.6 seconds (53.84 MB artifact).
- Headless Selenium Quality Gate: 88.2 seconds (3 full journeys).
- Docker Build & Push: 11.5 seconds (multi-stage cached build).
- Ansible Provision & Deploy: 76.2 seconds (remote node swap).
- Node Verification & Promotion: 8.7 seconds (health check & promote :stable).
- Total End-to-End Lead Time: 3 minutes, 35 seconds.

System Availability & Recovery Metrics

- Container Crash Recovery (systemd): 17 seconds downtime.
- Target Node Reboot Recovery: 22 seconds to application availability.
- Automated Rollback Latency: < 48 seconds from failure detection.
- Idempotency Metric: changed=0 across all 37 evaluation tasks.
- Total Releases Published: v1.0.0 (MVP) and v1.1.0 (Final DevOps).

Operational Challenges Overcome & Engineering Lessons

Real Troubleshooting Challenges

• Parallel Merge Conflicts: Resolved navbar conflicts in Week 6 via manual union.

• Spring Security Health Probes: Allowed unauthenticated /health access.

• Headless Chrome CI Crashes: Added --no-sandbox & --disable-dev-shm-usage.

• Docker Non-Root Permissions: Added --chown=appuser:appgroup in Dockerfile.

• Windows Jenkins Daemon Access: Configured Docker TCP daemon on port 2375.

• Firewall Lockout Guard: Explicitly permitted SSH port 22 before enabling UFW.

Core DevOps Lessons Learned

• Fast Feedback is Vital: Separate unit tests from slow browser tests with Maven profiles.

• Automated Quality Gates Work: Selenium prevents defect leakage before artifact push.

• Idempotency Prevents Drift: Declarative Ansible eliminates 'works on my machine'.

• Resilience Must Be Automated: systemd + block/rescue rollback guarantees uptime.

System Limitations, Future Roadmap & Project Deliverables

Current Limitations & Future Roadmap

- Database Persistence: Migrate in-memory H2 to PostgreSQL with Liquibase (P0).
- Transport Security: Deploy Nginx reverse proxy with Let's Encrypt TLS 1.3 (P0).
- Secrets Management: Encrypt configs using Ansible Vault and HashiCorp Vault (P0).
- Production Telemetry: Prometheus metric scraping and Grafana dashboards (P1).
- Orchestration: Kubernetes (K3s) deployment with Horizontal Pod Autoscaling (P2).

Final Deliverables & Repository Links

- GitHub Repository: ganesh-dahiphale/construction_project_dashboard.
- Production Releases: v1.0.0 (MVP) and v1.1.0 (End-to-End DevOps).
- Full Documentation: ARCHITECTURE, TROUBLESHOOTING, DEMO_SCRIPT, VIVA_PREP.
- Master Evidence Matrix: docs/EVIDENCE_INDEX.md linking all weekly logs.
- Thank You! Ready for questions and live demonstration.